

Machine Learning – HW#4

General notes:

- It is **recommended** that you work alone to learn the material. However, groups of size 2 are allowed.
- Read and understand the questions, and answer accordingly. Be sure to answer what is asked. When in doubt, you can provide more details.
- Some of the questions require material not covered in class, or will be covered in future lectures. This is intentional.
- **Warning:** do not copy other student work. This includes code similarity. Do not sit and solve assignments together, unless it is your group partner.

Submission document format guidelines:

- Submit pdf document to Gradescope.
- You can use word, latex, or any other editor.
- No need for cover page. Provide your names and IDs in the header.
- Answer each Question on a new page. No need to copy the question, just put Question # at the beginning, then a), b), etc. for each sub-question.
- When submitting to Gradescope, you will be given the chance to map pages to questions.
- For groups, one student submits and adds their teammate during submission on Gradescope.

Setup Keras

Note: you are not required to show anything for this setup part in your document submission.

For this homework we will use [Keras](#) package due to its ease-of-use. Keras now supports several backends (tensorflow, pytorch, etc.). Since we already have pytorch in the course's environment (ml557_env.yml), you can easily setup Keras with pytorch backend.

Here is easiest way to do it.

- Open anaconda command prompt, and activate the ml557 environment.
- Then install keras package:

```
conda install conda-forge::keras
```

- Now check if keras installation is okay by running python in the command line, and run the following statements.

```
import os
os.environ["KERAS_BACKEND"] = "torch"
import keras
```

- If you did not get any errors, then you've installed keras fine. Also note the three lines of code above. When using keras in your code, you need to setup the backend first as pytorch (if you are using pytorch).
- If you run into errors, try searching on google for the issues.

Question 1 [6+6+8=20 pts]

You are provided with 'hw4_q1.csv' of a simple dataset with one feature 'feat1' and regression label 'y'. **Treat this entire dataset as a training set.**

NOTE1: for each of the following parts show the *relevant* code *snippet*, plots, and results.

NOTE2: Even with seed setting, keras can give different results. So, you may not get the same result as me, that's fine. Just be sure to follow the instructions.

a) Plot the dataset points using matplotlib or seaborn as a scatterplot. The x-axis should be feat1 and y-axis is the label y. You should see that the dataset is non-linear (quadratic and sinusoidal).

b) Do the following using keras:

- Fix the random seed using: `keras.utils.set_random_seed(11)`
- Setup a three hidden layer neural network with **linear** activations. Use any optimizer you'd like with appropriate learning rate (Adam is recommended). Use MSE loss. Compile the model, and show the model summary.

Note: there are multiple ways to do the above in Keras. Use whichever you prefer. See this [keras guide](#). Also, in Keras you do not need to add a bias column, it does it for you.

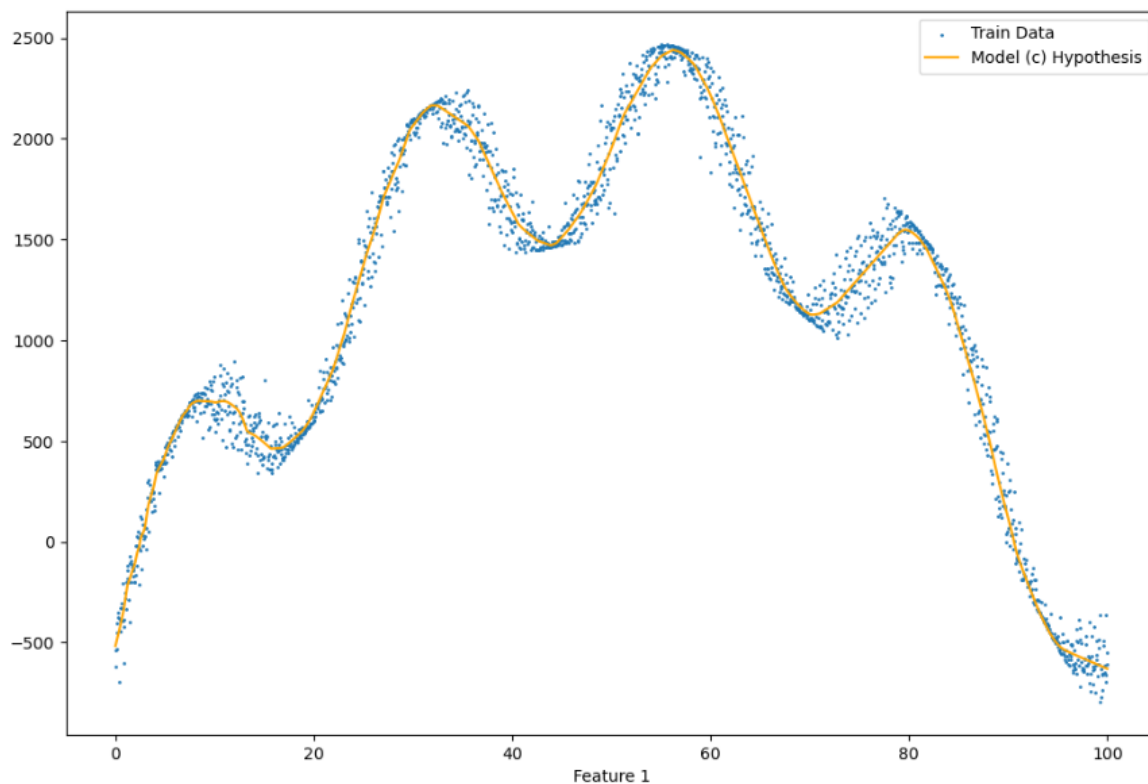
- Practice (no need to submit answer in document): How many parameters (weights) does this network have (calculate it yourself)? Your calculation should agree with results of `model.summary()`.
- Train the model using fit function. Use `batch_size=512` and `epochs=1000`.
- Plot the dataset points (scatter), and on top of it the learned hypothesis (plot).

Note: From the dataset, feat1 has values ranging from 0 to 100. To get points on the learned hypothesis, you can sample values from 0 to 100 using `np.linspace(0, 100, num=1000)`, then use `model.predict(X_samples)` to get predicted y-values. Then you can use seaborn lineplot or matplotlib plot to plot the hypothesis curve.

The plot should show that the hypothesis is linear since we did not use non-linear activations.

c) For this part, develop a neural network that can get good fit on this dataset. It is up to you to design the architecture, and the fit arguments. To confirm, plot the hypothesis decision to confirm that it follows the non-linear curve target.

Note: there is noise that I added to this dataset that may require a **considerably** large network to get perfect fit. However, the goal here is to get a model to fit the non-linear trend as shown in the plot below. So, as you play around with the network, visually inspect the fit you get.



Question 2 [6+4+6+4]=20 pts]

You are given the stroke **classification** dataset. You worked with this dataset before in previous homework. Here the dataset has been pre-processed for you, and you are given 'hw4_q2_train.csv', 'hw4_q2_val.csv', and 'hw4_q2_test.csv'.

The **label stroke** is the last column (binary {0, 1}), all else are features. **Notice that this dataset is skewed (class-1 is very rare).**

NOTE1: for each of the following parts show the *relevant* code *snippet*, plots, and results.

NOTE2: we will use train-val-test model selection scheme. Even with seed setting, keras can give different results. So, you may not get the same result as me, that's fine. Just be sure to follow the instructions.

a) Setup the following four Keras neural networks:

- Fix the random seed using: `keras.utils.set_random_seed(11)`
- Single-layer: a single output layer with sigmoid unit.
- 1H: a 1-hidden layer with 64 relu units, and an output layer with sigmoid unit.
- 2H: hidden layer with 64 relu units, following by hidden layer with 64 relu units, and an output layer with sigmoid unit.
- 3H: hidden layer with 64 relu units, following by hidden layer with 64 relu units, followed by hidden layer with 64 relu units, and an output layer with sigmoid unit.

For all these networks, use Adam optimizer with `learning_rate=0.001`. Use 'binary_crossentropy' loss. Compile these networks. Display the `summary()`.

Practice (no need to submit answer in document): calculate the parameters of these network yourself (it should agree with `summary()` results)

b) Do the following:

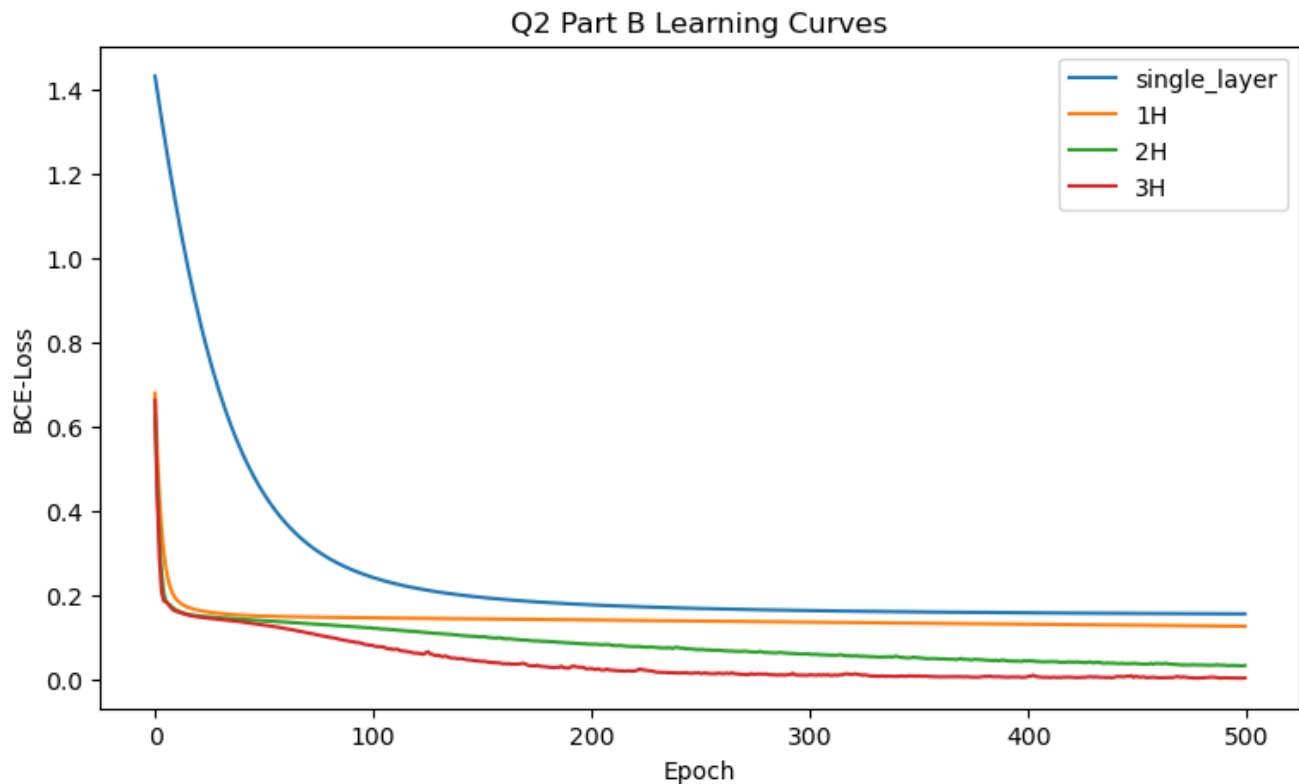
- Train these networks on the training set. Use `batch_size=512` and `epochs=500`.
- Plot the learning curves for all four networks on same plot. Label x and y axes appropriately. Use legend to name the curves. See note below and see my plot.

Note: you can get the loss per epoch from Keras history object returned by fit function.

```
history = nn_model.fit(X_train, y_train, batch_size=512, verbose=0, epochs=500)
losses = history.history['loss']
```

`losses` is a list of size `epochs`. You can then plot this using `seaborn` or `matplotlib`.

The following is the plot I got for this part (you may have different results).



c) Using the trained models, evaluate them on the validation set ROC and PR curves. You can do this using

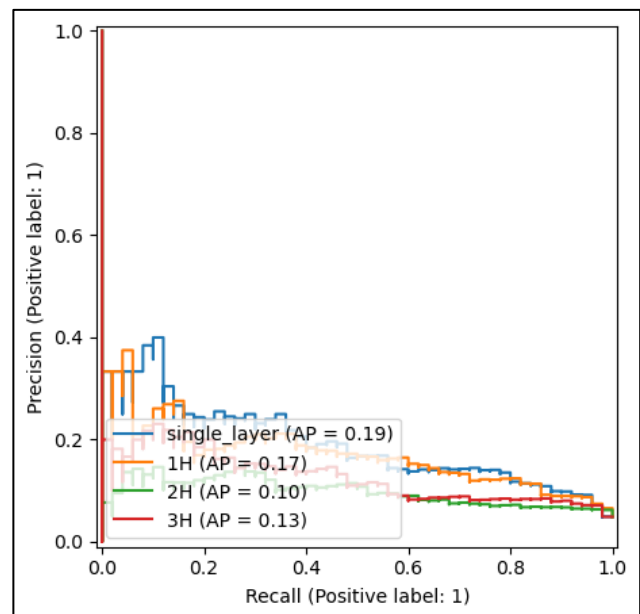
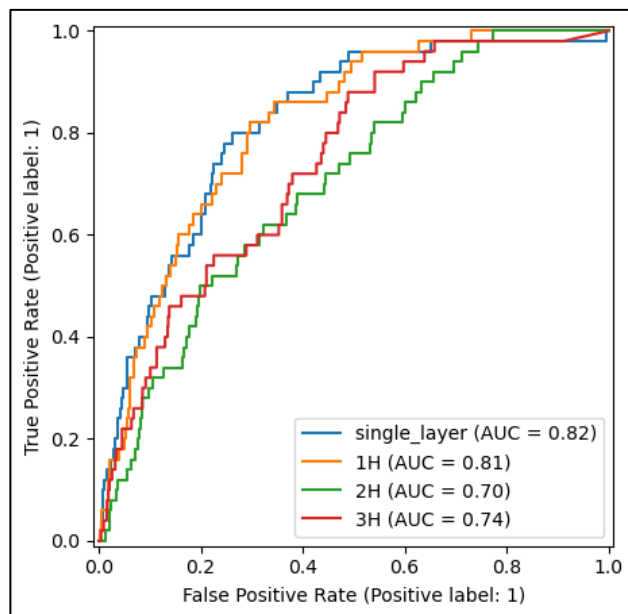
[`RocCurveDisplay.from\_predictions`](#) and [`PrecisionRecallDisplay.from\_predictions`](#)

Make one plot of ROC curves for all four models, and one plot of PR curves for all four models. Below shows example of how to plot ROC curves for two models A and B in same plot.

```
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(111)
```

```
RocCurveDisplay.from_predictions(y_val, y_val_A_preds, name='Model A', ax=ax)
RocCurveDisplay.from_predictions(y_val, y_val_B_preds, name='Model B', ax=ax)
```

The following are the curves I got.



- d) Select the best network from part (c) based on PR-AUC (because of the class skew).
- Merge the training set and validation sets into one. Numpy's [vstack](#) and [hstack](#) are helpful for this.
 - *Re-train* the best network on the merged dataset. **Note** you need to re-compile the network to reset the weights. Use Adam optimizer with `learning_rate=0.0001`, `batch_size=512`, and `epochs=500`.
 - Report ROC-AUC and PR-AUC performance on test set. Use scikit-learn functions [link](#) and [link](#).